

ECE0202 Lecture 19 中文逐页讲义

Universal Asynchronous Receiver and Transmitter (UART)

面向完全初学者的直白解释版

本讲主线：UART 是一种常见的串口通信方式。它让两个设备用少数几根线发送和接收二进制数据。本讲要掌握五条线索：**UART 是什么**，**data frame 怎么组织**，**baud rate 怎么算**，**STM32 如何配置 UART 引脚和寄存器**，以及 **polling / interrupt 的接收发送方式有什么区别**。

Contents

Slide 1: 课程标题页	2
Slide 2: Universal Asynchronous Receiver and Transmitter (UART)	2
Slide 3: RS232	2
Slide 4-5: Connecting to PC 与 FT232R	3
Slide 7: Data Frame	3
Slide 8: Error Detection	4
Slide 9: Transmitting 0x32 and 0x3C	5
Slide 10: Baud Rate 与真实数据吞吐率	6
Slide 11: Baud Rate 公式、OVER8 与 BRR	6
Slide 12: Baud Rate Example 1: 16 MHz, 14400	7
Slide 13: Baud Rate Example 2: 80 MHz, 9600	7
Slide 14: UART Connection	8
Slide 15: Setting Up the Code: USART 初始化函数	8
Slide 16-18: GPIO 引脚、Alternate Function 与 UART 时钟配置	9
Slide 19: UART Receive by Polling	11
Slide 20: UART Send by Polling	11
Slide 21: UART Interrupt: Receiving Data 概念图	12
Slide 22: UART Interrupt: Receiving Data 代码	13
Slide 23: UART Interrupt: Sending Data	14
Slide 27: ASCII	14

Slide 28: Voltage Levels: RS-232 / RS-422 / RS-485	15
Lecture 19 总结	16
容易混淆点清单	16
一页复习版	16

Slide 1: 课程标题页

这一页只是课程封面，说明本讲属于 ECE0202 Embedded Processors & Interfacing + Lab。

这一讲的主题是 UART。不要把 UART 当成一个抽象名词背下来。它本质上是：**两个数字设备之间传输 0 和 1 的一种规则**。规则包括：什么时候开始、每一位持续多久、数据有几位、是否校验、什么时候结束。

一句话：UART 是嵌入式系统里最常见的串口通信方式之一，常用于 STM32 和电脑、传感器、蓝牙模块、GPS 模块之间通信。

Slide 2: Universal Asynchronous Receiver and Transmitter (UART)

这一页给出 UART 的全称：**Universal Asynchronous Receiver and Transmitter**（通用异步收发器）。

1. Universal 是什么？

Universal（通用）的意思是：UART 不是某一个专用设备的私有协议。很多芯片、模块和标准都支持类似 UART 的串口通信方式。比如 STM32、Arduino、蓝牙串口模块、GPS 模块、USB-to-UART 模块都可以通过 UART 思想传输数据。

2. Asynchronous 是什么？

Asynchronous（异步）是本页最关键的词。它的意思是：**发送方不会额外给接收方一根 clock 时钟线**。

对比一下：

通信方式	是否有专门 clock 线	典型例子
Synchronous 同步通信	有	SPI, I2C 的 SCL/SCK
Asynchronous 异步通信	没有	UART

UART 没有 clock 线，所以发送方和接收方必须提前约定好传输速度，也就是后面反复讲的 **baud rate**（波特率）。如果两边速度不一致，接收方就会在错误的时间点采样，数据会读错。

3. Receiver 和 Transmitter 是什么？

Receiver（接收器）负责收数据，**Transmitter**（发送器）负责发数据。UART 通常有两根核心数据线：

- TX: transmit, 发送线。
- RX: receive, 接收线。

两个设备连接时，一般是交叉连接：设备 A 的 TX 接设备 B 的 RX，设备 A 的 RX 接设备 B 的 TX。

易错点：UART 的“异步”不是说两边随便发。恰恰相反，因为没有 clock 线，两边必须更严格地使用相同 baud rate 和相同 frame 格式。

Slide 3: RS232

这一页介绍 RS-232。初学者最容易把 **UART** 和 **RS-232** 混为一谈，这里必须分开。

1. UART 和 RS-232 的关系

名称	本质
UART	一种串口通信逻辑：怎么发 start bit、data bits、parity、stop bit
RS-232	一种物理电气标准：电压范围、接口、传输距离等

UART 更像“语言语法”，RS-232 更像“电压和插头规格”。

STM32 内部 UART 引脚通常使用 0 V 表示逻辑 0，3.3 V 表示逻辑 1。传统 RS-232 的电压不是这样，通常逻辑 1 和逻辑 0 使用正负电压。正因为电压不兼容，STM32 不能直接硬接传统 RS-232 口。

2. 为什么这一页出现老式串口图？

图中 RS-232 接头是以前电脑常见的 DB9 串口。现在电脑通常没有这个接口，而是 USB。因此我们常用 USB-to-UART 转换芯片，例如 FT232R，把 STM32 的 UART 转成电脑能识别的 USB 串口。

一句话：UART 是通信逻辑，RS-232 是物理电压标准；STM32 的 UART 不能直接等同于 RS-232。

Slide 4-5: Connecting to PC 与 FT232R

这两页讲 STM32 如何连接到电脑。

1. 为什么需要 FT232R？

电脑通常只有 USB 口，但 STM32 的 UART 是 TX/RX 串口信号。USB 和 UART 不是同一种通信方式，不能直接相连。

FT232R（USB-to-UART 转换芯片）的作用是：

- STM32 侧：提供 UART 的 TX, RX, GND。
- 电脑侧：变成 USB 设备，电脑会把它识别成一个串口 COM port。

2. 接线逻辑

常见连接方式：

STM32	FT232R	含义
TX	RX	STM32 发出的数据给 FT232R 接收
RX	TX	FT232R 发出的数据给 STM32 接收
GND	GND	两边必须共地

注意 TX 和 RX 要交叉，不是 TX 对 TX。发送线要接到对方接收线。

易错点：只接 TX/RX 不接 GND 很容易通信失败。GND 是电压参考点。没有共同参考点，3.3 V 对另一边来说可能根本不是同一个 3.3 V。

Slide 7: Data Frame

这一页是 UART 的核心之一：**Data Frame**（数据帧）。

1. 为什么需要 frame?

UART 没有时钟线，所以接收方必须知道：一个字节从哪里开始、哪里是数据、哪里结束。Data frame 就是把一段数据包装成一个固定格式。

典型 UART frame:

Idle	Start bit	Data bits	Optional parity	Stop bit
空闲高电平	起始位	数据位	可选校验位	停止位

2. Start bit 起始位

UART 空闲时线路通常保持高电平。发送开始时，发送方把线拉低一位时间。接收方看到从高到低的变化，就知道：一帧开始了。

3. Data bits 数据位

数据位通常是 7、8 或 9 位。最常见的是 8 位。数据可以 **LSB first**（最低有效位先发），也可以 **MSB first**，但 UART 常见配置是 **LSB first**。

4. Parity bit 校验位

Parity（奇偶校验）是一个简单错误检测方法。它不能纠错，只能发现一部分错误。

5. Stop bit 停止位

Stop bit 通常是高电平，用来说明这一帧结束。常见是 1 stop bit，也可以配置为 2 stop bits 等。

6. Tolerate 10% clock shift 是什么意思?

UART 两边没有共同 clock，所以接收方靠自己的时钟估计每个位的中间位置。如果发送方和接收方 clock 有一点偏差，UART 仍然能容忍一定误差。PPT 写 10% 是强调：UART 不是完全不允许误差，但误差过大会读错。

一句话：UART frame 是“起始位 + 数据位 + 可选校验位 + 停止位”的组合，用来让接收方知道一段串行数据如何解析。

Slide 8: Error Detection

这一页讲 parity 奇偶校验。

1. Even Parity 偶校验

Even Parity（偶校验）的规则是：数据位和 parity bit 加起来，1 的总数必须是偶数。

例子：数据为 10101011。数一下有几个 1：

$$1 + 0 + 1 + 0 + 1 + 0 + 1 + 1 \Rightarrow 5 \text{ 个 } 1$$

5 是奇数。如果使用 even parity，就要让总数变成偶数，所以 parity bit 应该是 1。这样总共有 6 个 1。

2. Odd Parity 奇校验

Odd Parity（奇校验）的规则是：数据位和 parity bit 加起来，1 的总数必须是奇数。同样数据 10101011 已经有 5 个 1，是奇数，所以 odd parity 下 parity bit 应该是 0。

数据	校验方式	parity bit
10101011, 已有 5 个 1	Odd parity	0
10101011, 已有 5 个 1	Even parity	1

3. 它能检测什么错误?

如果传输过程中有 1 bit 翻转，1 的个数奇偶性会改变，所以接收方能发现错误。

但是，如果两个 bit 同时翻转，奇偶性可能不变，parity 就检测不出来。

易错点：Parity 只能做简单错误检测，不能告诉你哪一位错了，也不能自动修复数据。

Slide 9: Transmitting 0x32 and 0x3C

这一页用具体波形说明 UART 如何发送两个字节：0x32 和 0x3C。

1. 配置是什么?

PPT 标出：

- 1 start bit: 1 个起始位。
- 8 data bits: 8 个数据位。
- no parity: 无校验位。
- 1 stop bit: 1 个停止位。
- baud rate = 9600: 每秒传 9600 个 bit 时间。

这通常写成 **8N1** (8 data bits, No parity, 1 stop bit)。

2. 0x32 为什么在波形里不是直接写成 00110010?

0x32 的二进制是：

$$0x32 = 0011\ 0010_2$$

UART 常见是 LSB first, 即最低位先发。按从低位到高位读，发送顺序是：

$$0, 1, 0, 0, 1, 1, 0, 0$$

所以你看波形时，不能按普通二进制从左到右直接读成 00110010，必须考虑 LSB first。

3. 每一位持续多久?

baud rate = 9600, 表示每秒 9600 bit 时间。所以一位持续时间为：

$$T_{bit} = \frac{1}{9600} \approx 0.000104\ s = 0.104\ ms$$

4. idle / start / stop 的电平

常见 UART 逻辑：

- Idle 空闲：高电平。
- Start bit：低电平。
- Data bits：按数据位高低输出。
- Stop bit：高电平。

一句话：UART 发送一个字节不是只发 8 位数据，还要加 start bit 和 stop bit；并且数据常见是 LSB first。

Slide 10: Baud Rate 与真实数据吞吐率

这一页讲 **Baud Rate**（波特率）。

1. Baud rate 是什么？

在本课程语境下，baud rate 可以直白理解为：**每秒传输多少个 bit 时间**。

例如 baud rate = 9600:

每秒传9600 个 bit slot

2. 为什么 9600 baud 不等于 1200 bytes/s?

如果只看 8 bit = 1 byte，会以为：

$$9600/8 = 1200 \text{ bytes/s}$$

但这是错的，因为 UART 一帧不只有 8 data bits。以 8N1 为例：

$$1 \text{ start} + 8 \text{ data} + 1 \text{ stop} = 10 \text{ bits}$$

真正有效数据吞吐率：

$$9600/(1 + 8 + 1) = 960 \text{ bytes/s}$$

3. Protocol overhead 协议开销

Start bit 和 stop bit 不属于实际数据，但必须传输，这就是 **protocol overhead**（协议开销）。

易错点：Baud rate 是线路上的 bit 时间速度，不等于实际有效数据字节速度。真实数据速度必须扣掉 start、stop、parity 等开销。

Slide 11: Baud Rate 公式、OVER8 与 BRR

这一页是寄存器配置的核心。

公式：

$$\text{Baud Rate} = \frac{(1 + \text{OVER8}) \times f_{PCLK}}{USARTDIV}$$

1. 各变量含义

符号	含义
f_{PCLK}	UART 使用的外设时钟频率，例如 16 MHz 或 80 MHz
OVER8	过采样设置。0 表示 oversampling by 16, 1 表示 oversampling by 8
USARTDIV	波特率分频值，写在 BRR 寄存器里
BRR	Baud Rate Register, 波特率寄存器

2. Oversampling 过采样是什么？

Oversampling（过采样）的意思是：接收方不会只在每个 bit 随便采一次，而是用更高频率多次采样，估计 bit 中心点，提升可靠性。

- OVER8 = 0: 16 倍过采样，更稳，常用。
- OVER8 = 1: 8 倍过采样，可以支持更高 baud rate，但抗误差能力弱一些。

3. BRR 的作用

BRR 不是直接写 baud rate，而是写分频值。UART 用这个分频值把外设时钟变成目标传输速度。

一句话：设置 baud rate 的本质是：根据 UART 时钟和目标 baud rate 算出 USARTDIV，然后把这个值写进 BRR。

Slide 12: Baud Rate Example 1: 16 MHz, 14400

题目：已知 OVER8=0, processor clock = 16 MHz, 目标 baud = 14400, 求 USARTDIV / BRR。

因为 OVER8=0, 公式变成：

$$USARTDIV = \frac{f_{PCLK}}{Baud\ Rate}$$

代入：

$$USARTDIV = \frac{16,000,000}{14,400} \approx 1111.11$$

PPT 取近似：

$$USARTDIV \approx 1111 = 0x457$$

为什么可以近似？

寄存器不能存无限小数，实际配置时只能写可表示的整数或固定格式。近似会带来很小的 baud rate 误差，只要误差在接收容忍范围内，通信就能正常。

易错点：实际工程中通常还要计算 baud rate error。如果误差太大，接收端会采样错位。

Slide 13: Baud Rate Example 2: 80 MHz, 9600

题目：已知 OVER8=0, processor clock = 80 MHz, 目标 baud = 9600。

公式：

$$USARTDIV = \frac{80,000,000}{9600} \approx 8333.33$$

PPT 给出:

$$USARTDIV \approx 8333 = 0x208D$$

所以代码里可以看到:

```
USARTx->BRR = 0x208D;
```

这行不是随便写的。它来自: 80 MHz UART clock、9600 baud、OVER8=0 的计算。

一句话: BRR=0x208D 的含义是“用当前 UART 时钟生成约 9600 baud 的通信速度”。

Slide 14: UART Connection

这一页画的是两个 STM32L4 或两个 UART 端口之间的连接。

1. TX 和 RX 必须交叉

连接规律:

设备 A	设备 B	原因
TX	RX	A 发出的数据要进入 B 的接收端
RX	TX	B 发出的数据要进入 A 的接收端
GND	GND	两边必须共地

2. UART 是全双工吗?

典型 UART 有独立 TX 和 RX 两根数据线, 因此可以 **full-duplex** (全双工): 一边发送的同时也能接收。但不能在软件上同时处理好, 要看你用 polling、interrupt 还是 DMA。

3. 为什么不能 TX 接 TX?

TX 是输出端, 两个输出端硬接可能互相冲突; 正确逻辑是“你的输出进我的输入”。

易错点: UART 接线失败最常见原因: TX/RX 没交叉、GND 没共地、baud rate 不一致、frame 格式不一致。

Slide 15: Setting Up the Code: USART 初始化函数

这一页是 USART_Init() 函数, 配置 USART 本体寄存器。

核心代码结构:

```
void USART_Init(USART_TypeDef * USARTx) {
    USARTx->CR1 &= ~USART_CR1_UE;           // Disable USART
    USARTx->CR1 &= ~USART_CR1_M;             // 8 data bits
    USARTx->CR2 &= ~USART_CR2_STOP;          // 1 stop bit
    USARTx->CR1 &= ~USART_CR1_PCE;           // no parity
    USARTx->CR1 &= ~USART_CR1_OVER8;         // oversampling by 16
    USARTx->BRR = 0x208D;                     // baud rate 9600 when clock is 80 MHz
    USARTx->CR1 |= (USART_CR1_TE | USART_CR1_RE); // enable TX and RX
    USARTx->CR1 |= USART_CR1_UE;             // enable USART
    while ((USARTx->ISR & USART_ISR_TEACK) == 0);
    while ((USARTx->ISR & USART_ISR_REACK) == 0);
}
```

```
}
```

1. 为什么先 **disable USART**?

改通信格式前先关掉 USART，避免运行中修改导致半帧数据错误。和你前面学 RTC Alarm 先 disable 一样，本质是：改正在工作的硬件前先停一下。

2. CR1 / CR2 是什么?

- CR1: Control Register 1, 控制 USART 的启用、数据长度、奇偶校验、过采样、发送接收使能等。
- CR2: Control Register 2, 常用于 stop bit 等更细配置。

3. M bits 数据长度

USART_CR1_M 控制数据长度。清掉它表示使用 8 data bits。不同 STM32 版本细节可能有 M0/M1 两个位，但课程这里简化为配置成 8-bit 数据。

4. STOP bits

清 USART_CR2_STOP 表示 1 stop bit。这对应前面讲的 8N1。

5. PCE parity control enable

清 USART_CR1_PCE 表示 no parity。

6. OVER8

清 USART_CR1_OVER8 表示 oversampling by 16。

7. TE / RE / UE

位	英文	作用
TE	Transmitter Enable	允许发送
RE	Receiver Enable	允许接收
UE	USART Enable	启用整个 USART 模块

8. TEACK / REACK

TEACK 和 REACK 是硬件确认标志。代码等待它们变为 1，表示发送器和接收器真的准备好了。

一句话：USART 初始化就是把通信双方约定的规则写进寄存器：8 data bits、1 stop bit、no parity、9600 baud、允许收发、启用 USART。

Slide 16-18: GPIO 引脚、Alternate Function 与 UART 时钟配置

这几页解释代码如何把普通 GPIO 引脚变成 UART 的 TX/RX 引脚。

1. 为什么 GPIO 要配置 Alternate Function?

STM32 的一个物理引脚可以有多种功能。例如 PB6 可以当普通 GPIO，也可以当 USART1_TX。要让它连接到 UART 外设，必须把引脚模式设成 **Alternate Function**（复用功能）。

GPIO 模式通常是：

MODER 位值	模式	含义
00	Input	输入
01	Output	普通输出
10	Alternate Function	外设复用功能
11	Analog	模拟模式

2. PB6 / PB7 用作 USART1

PPT 代码里：

- PB6 = AF7，作为 USART1_TX。
- PB7 = AF7，作为 USART1_RX。

这里 AF7 是 alternate function 编号，不是 interrupt number，也不是 GPIO pin number。

3. AFR 寄存器是什么?

AFR = Alternate Function Register。它决定某个引脚在 alternate function 模式下连接到哪个外设功能。

例如：

```
GPIOB->AFR[0] |= 0x77 << (4*6); // Set pin 6 and 7 to AF7
```

每个 pin 的 AF 编号通常占 4 bits。PB6 和 PB7 写成 7，表示 AF7。

4. OSPEEDR / PUPDR / OTYPER

寄存器	作用
OSPEEDR	输出速度。UART 边沿较快，常设置 high speed。
PUPDR	上拉/下拉。UART RX 常需要稳定空闲状态，课程代码选择 pull-up。
OTYPER	输出类型。UART TX 常用 push-pull。

5. UART 外设时钟

除了 GPIO 时钟，还要打开 UART 外设本身的 clock。例如：

```
RCC->APB2ENR |= RCC_APB2ENR_USART1EN; // Enable USART1 clock
RCC->APB1ENR1 |= RCC_APB1ENR1_UART4EN; // Enable UART4 clock
```

没开 clock，外设寄存器配置可能无效。STM32 里“先开时钟再配寄存器”是固定习惯。

6. CCIPR 选择 UART clock source

RCC->CCIPR 可以选择 USART 的时钟来源，例如 PCLK、SYSCLK、HSI16、LSE。选择不同源会影响 baud rate 计算，因为公式里的 f_{PCLK} 或 UART clock 不同。

易错点：配置 UART 不是只写 USART 寄存器。你还必须配置 GPIO alternate function、GPIO speed/pull-up/output type、外设时钟、UART clock source。

Slide 19: UART Receive by Polling

这一页讲 **Polling**（轮询）方式接收数据。

典型代码：

```
void USART_Read(USART_TypeDef *USARTx, uint8_t *buffer, uint32_t nBytes) {
    int i;
    for (i = 0; i < nBytes; i++) {
        while (!(USARTx->ISR & USART_ISR_RXNE)); // wait until RXNE is set
        buffer[i] = USARTx->RDR;                 // reading RDR clears RXNE
    }
}
```

1. RXNE 是什么？

RXNE (Receive Data Register Not Empty, 接收数据寄存器非空)。当硬件收到一个字节并放到 RDR 里时, RXNE 置 1。

2. RDR 是什么？

RDR = Receive Data Register, 接收数据寄存器。CPU 从这里读出收到的字节。

3. while 循环在等什么？

```
while (!(USARTx->ISR & USART_ISR_RXNE));
```

这行意思是：只要 RXNE 还没置位，就一直原地等。等到硬件收到数据，RXNE=1，才继续读 RDR。

4. 为什么说 polling 低效？

CPU 在 while 里什么都不干，只是在不断问：“数据来了吗？”这就是 busy waiting。数据没来时，CPU 时间被浪费。

一句话：Polling 接收简单，但 CPU 会被卡在等待循环里，不能高效做其他任务。

Slide 20: UART Send by Polling

这一页讲轮询方式发送数据。

典型代码：

```
void USART_Write(USART_TypeDef *USARTx, uint8_t *buffer, uint32_t nBytes) {
    int i;
    for (i = 0; i < nBytes; i++) {
        while (!(USARTx->ISR & USART_ISR_TXE)); // wait until TXE is set
    }
}
```

```
    USARTx->TDR = buffer[i] & 0xFF;           // writing TDR clears TXE
}
while (!(USARTx->ISR & USART_ISR_TC));        // wait transmission complete
    USARTx->ICR |= USART_ICR_TCCF;            // clear TC flag
}
```

1. TXE 是什么?

TXE (Transmit Data Register Empty, 发送数据寄存器空)。它表示 TDR 现在可以写入新字节。

2. TDR 是什么?

TDR = Transmit Data Register, 发送数据寄存器。CPU 把要发送的字节写到 TDR, UART 硬件再把它移入 shift register, 一位一位发到 TX 线上。

3. TC 是什么?

TC (Transmission Complete, 发送完成)。它表示最后一个字节不仅写进 TDR, 而且已经真正从 TX 线上发送完成。

4. 为什么要清 TCCF?

TCCF = Transmission Complete Clear Flag。向它写 1 可以清除 TC 标志。

易错点: TXE 只表示 TDR 可以写下一个字节, 不一定表示整个发送完成。TC 才表示传输完成。

Slide 21: UART Interrupt: Receiving Data 概念图

这一页开始从 polling 过渡到 **Interrupt** (中断) 接收。

1. Polling 的问题

Polling 需要 CPU 一直检查 RXNE。数据没来时 CPU 也被 while 占住。

2. Interrupt 的思路

中断方式是:

1. CPU 平时执行主程序。
2. UART 收到一个字节。
3. 硬件把字节放入 RDR, 并设置 RXNE。
4. 如果 RXNE interrupt 使能, NVIC 让 CPU 进入 USART IRQHandler。
5. ISR 读取 RDR, 把数据存进 buffer。
6. ISR 结束, CPU 回主程序。

3. 图里的硬件路径

图中可以理解成：

模块	作用
RX pin	外部串行数据进入 STM32 的引脚
RX Shift Register	把一位一位进来的串行数据拼成一个字节
RDR Register	接收数据寄存器，CPU 最终从这里读字节
RXNE Flag	RDR 非空，表示有数据可读
Interrupt Controller	如果中断允许，就触发 CPU 执行 ISR

一句话：UART interrupt 接收就是“数据来了再打断 CPU”，比 polling 更省 CPU 时间。

Slide 22: UART Interrupt: Receiving Data 代码

这一页给出接收中断的代码。

```
void USART_IRQHandler(USART_TypeDef * USARTx,
                      uint8_t * buffer,
                      uint8_t * pRx_counter) {
    if (USARTx->SR & USART_SR_RXNE) { // Received data
        buffer[*pRx_counter] = USARTx->DR;
        // Reading USART_DR automatically clears the RXNE flag
        (*pRx_counter)++;
        if ((*pRx_counter) >= BufferSize)
            (*pRx_counter) = 0;
    }
}

void USART1_IRQHandler(void) {
    USART_IRQHandler(USART1, USART1_Buffer_Rx, &Rx1_Counter);
}

void USART2_IRQHandler(void) {
    USART_IRQHandler(USART2, USART2_Buffer_Rx, &Rx2_Counter);
}
```

1. 为什么先检查 RXNE?

一个 USART 中断入口可能由不同事件触发。进入 handler 后要先确认是否真的因为收到数据：

```
if (USARTx->SR & USART_SR_RXNE)
```

如果 RXNE=1，说明有数据可读。

2. 为什么读 DR 会自动清 RXNE?

硬件认为：你已经把数据读走了，RDR 不再“非空”，所以 RXNE 自动清零。这是很多状态标志的典型清除方式。

3. buffer 和 counter 在干什么?

buffer 是接收缓冲区，用来存收到的字节。pRx_counter 指向当前写入位置。每收到一个字节，写入 buffer，然后 counter 加 1。

如果 counter 到达 BufferSize, 就回到 0:

```
if ((*pRx_counter) >= BufferSize)
    (*pRx_counter) = 0;
```

这是一种简单 **circular buffer** (循环缓冲区) 思想。

4. 为什么 USART1 和 USART2 都调用同一个通用函数?

这是代码复用。USART1_IRQHandler 和 USART2_IRQHandler 只是把不同 USART 和不同 buffer 传给同一个处理函数。

易错点: ISR 里面不要做太久的事情。接收中断应尽快读出数据、存入 buffer, 然后退出。复杂处理留给主循环。

Slide 23: UART Interrupt: Sending Data

这一页讲中断方式发送数据。

1. 发送为什么也可以用中断?

Polling 发送要一直等 TXE: TDR 空了才能写下一个字节。中断方式的思路是: TDR 一空, 硬件触发 TXE interrupt, CPU 再写下一个字节。

2. TXE interrupt 的流程

1. 软件先准备一个发送 buffer。
2. 启用 TXE interrupt。
3. 当 TDR 空, TXE=1。
4. CPU 进入 USART ISR。
5. ISR 写一个字节到 TDR。
6. 如果还有字节, 等待下一次 TXE interrupt。
7. 如果全部发完, 关闭 TXE interrupt, 防止空转中断。

3. 为什么发完要 disable TXE interrupt?

TXE 的含义是 TDR 空。如果没有数据要发, TDR 会一直空。若 TXE interrupt 不关闭, CPU 会一直进中断, 造成中断风暴。

一句话: UART 发送中断是“发送寄存器空了再通知 CPU 填下一个字节”, 避免 CPU 在 while 里傻等。

Slide 27: ASCII

这一页讲 **ASCII** (American Standard Code for Information Interchange, 美国信息交换标准代码)。

1. ASCII 是什么?

UART 传输的本质是字节。电脑终端显示字符时，需要把字节解释成字符。ASCII 就是最基础的一套映射表。例如：

字符	十进制	十六进制
'A'	65	0x41
'a'	97	0x61
'0'	48	0x30
换行 LF	10	0x0A
回车 CR	13	0x0D

2. 为什么嵌入式常用 ASCII?

如果 STM32 通过 UART 发出字节 0x41，电脑串口助手如果按 ASCII 显示，就会显示字符 A。如果按十六进制显示，就会显示 41。

同一段数据，显示方式不同，看到的结果不同。

易错点：'1' 和数值 1 不同。'1' 的 ASCII 是 0x31，数值 1 是 0x01。

Slide 28: Voltage Levels: RS-232 / RS-422 / RS-485

最后一页比较不同串口物理层标准。

标准	电压信号	最大距离	最大速度	设备数量
RS-232	Single-ended, 逻辑 1: +3 到 +15 V, 逻辑 0: -3 到 -15 V	100 feet	115 Kbit/s	1 master, 1 receiver
RS-422	Differential, 约 -6 V 到 +6 V	4000 feet	10 Mbit/s	1 master, 10 receivers
RS-485	Differential, 约 -7 V 到 +12 V	4000 feet	10 Mbit/s	32 masters, 32 receivers

1. Single-ended 单端信号

Single-ended (单端) 是信号相对于 GND 测量。普通 STM32 GPIO/UART 也是类似思路：看某条线相对地是高还是低。缺点是长距离抗干扰弱。

2. Differential 差分信号

Differential (差分) 用两根线之间的电压差表示 0 和 1。外界噪声往往同时影响两根线，差值变化较小，因此长距离更稳定。

3. 为什么 RS-485 支持多个设备?

RS-485 设计上支持总线结构，多个设备挂在同一对差分线上，因此工业通信里很常见。

一句话：UART 是通信逻辑；RS-232、RS-422、RS-485 是不同物理层电气标准。距离越长、环境越吵，差分通信越有优势。

Lecture 19 总结

必须掌握的核心概念

1. **UART** 是异步串口通信。没有 clock 线, 必须约定 baud rate。
2. **UART frame** 不只有数据位。常见 $8N1 = 1 \text{ start} + 8 \text{ data} + 1 \text{ stop}$ 。
3. **Baud rate** 不等于有效字节速率。9600 baud 的 8N1 有效速率是 960 bytes/s。
4. **BRR** 存的是分频值。根据 clock、baud rate、OVER8 计算得到。
5. **GPIO** 必须配置成 **alternate function**。否则物理 pin 不会连接到 USART 外设。
6. **RXNE** 表示收到数据, **TXE** 表示发送寄存器空, **TC** 表示发送完成。
7. **Polling** 简单但低效, 中断更高效。DMA 最高效但本讲只初步涉及。
8. **ASCII** 是字节到字符的映射。'1' 不等于数值 1。
9. **UART** 与 **RS-232/RS-485** 不是同一个层次。UART 是逻辑协议, RS 标准是电气物理层。

容易混淆点清单

- **TX/RX 接线**: TX 接对方 RX, RX 接对方 TX, 还要共地。
- **baud rate**: 线路 bit 时间速度, 不是纯数据字节速度。
- **TXE vs TC**: TXE 表示 TDR 空, 可以写下一字节; TC 表示最后一位也发完。
- **RXNE 清除**: 读 RDR/DR 会清 RXNE。
- **BRR**: 不是写 9600, 而是写由公式算出来的分频值。
- **RS-232 电压**: 不能把 STM32 3.3 V UART 直接接传统 RS-232 电压接口。

一页复习版

UART 发送数据时, TX 线空闲为高, 先发 start bit 低电平, 然后按 LSB first 发数据位, 接着可选 parity bit, 最后发 stop bit 高电平。接收方没有时钟线, 所以必须提前知道 baud rate 和 frame 格式。STM32 配置 UART 时, 需要配置 GPIO alternate function、打开外设时钟、设置 BRR、设置数据长度/stop/parity/oversampling, 并启用 TE、RE、UE。接收数据时, RXNE=1 表示 RDR 里有数据; 发送数据时, TXE=1 表示 TDR 可以写, TC=1 表示发送完成。Polling 简单但浪费 CPU, 中断方式让数据到来或发送寄存器空时再通知 CPU。